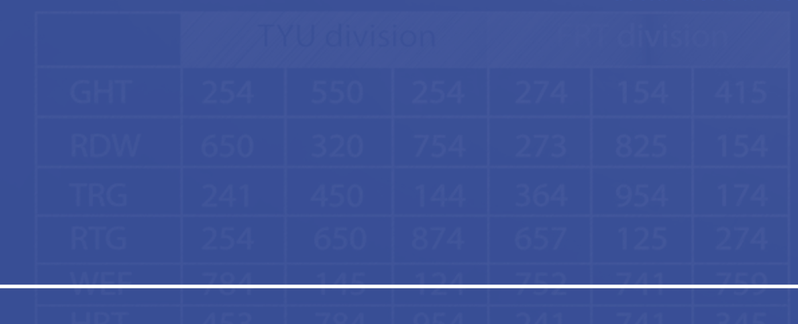
Applying BERT

MLP Lab.



This slide was adapted from the book "*Getting Started with Google BERT*"

Contents

Contents

01 Explore pre-trained BERT models

Q) Why use a pre-trained BERT??

A) Pre-training BERT from scratch is resource and time intensive.

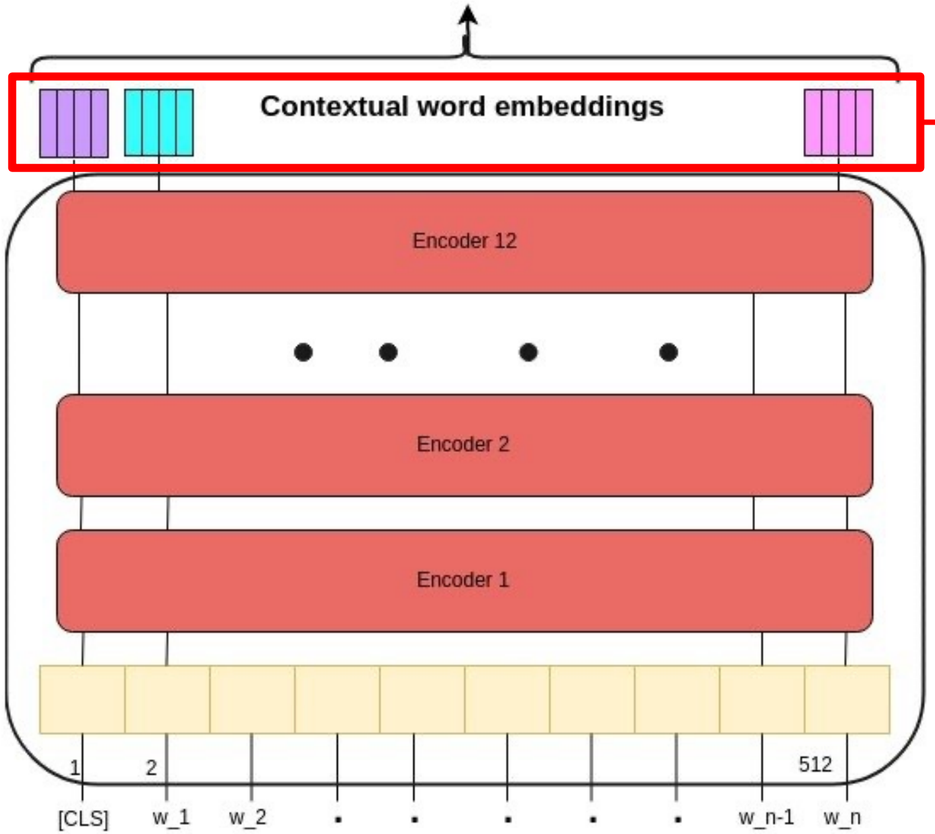
[Pre-trained BERT models provided by Google]

	H=128	H=256	H=512	H=768
L=2	2/128 (BERT-tiny)	2/256	2/512	2/768
L=4	4/128	4/256 (BERT-mini)	4/512 (BERT-small)	4/768
L=6	6/128	6/256	6/512	6/768
L=8	8/128	8/256	8/512 (BERT-medium)	8/768
L=10	10/128	10/256	10/512	10/768
L=12	12/128	12/256	12/512	12/768 (BERT-base)

Google Research's GitHub repository(https://github.com/google-research/bert)

02 Methods for Applying the pretrained BERT

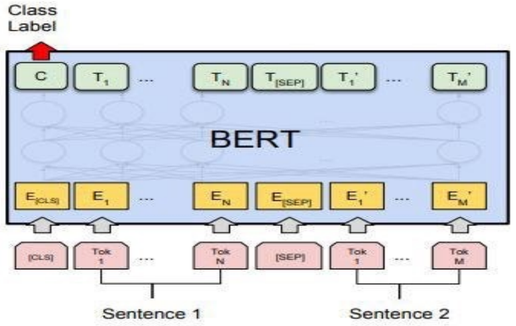
1. Extract embeddings and use them as word embeddings



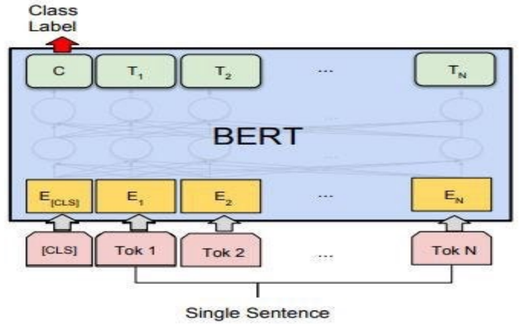
Use a pre-trained BERT model as a dynamic feature extractor

02 Methods for Applying the pretrained BERT

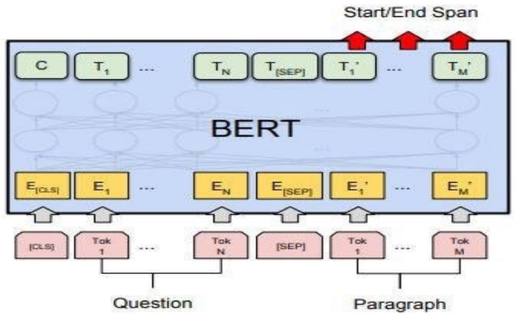
2. Fine-tune a pre-trained BERT model for downstream tasks



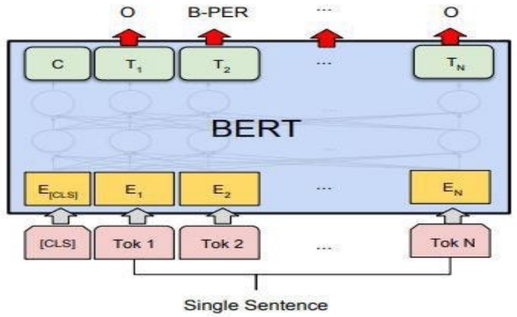
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA



(c) Question Answering Tasks: SQuAD v1.1



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Contents

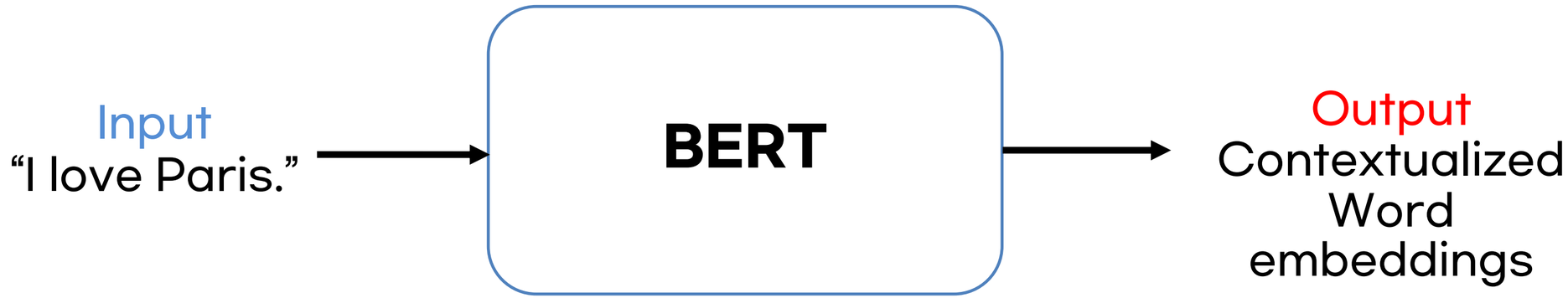
01 Embedding Extraction from BERT

- Example Sentence : “I love Paris.”

  ✓ Goal : Extract the contextual embedding of each word

  Q) Is it possible to get the embedding vector of each word by just inputting the sentence??

  A) NO, BERT requires some preliminary work



01 Embedding Extraction from BERT

- Four preprocessing steps are needed to get the BERT embedding

  1. Tokenization
  2. Add Special Tokens
  3. Add Attention Mask
  4. Token ID mapping

01 Embedding Extraction from BERT

- Preprocessing Step 1: Tokenization
  - ➢ Before a sentence can be fed into BERT, it must be tokenized.

[Tokenization]

01 Embedding Extraction from BERT

- Preprocessing Step 2 : Add Special Tokens

  ➢ Special tokens such as [CLS], [SEP], and [PAD] should be added to the tokenized sentence.

  ✓ [CLS] : A start token for sentences

  ✓ [SEP] : End tokens for each sentence (used to separate two or more sentences)

  ✓ [PAD] : A token used to equalize the length of tokens

  [Add Special Tokens]

  [I,love,Paris] —Add [CLS],[SEP],[PAD]→ [[CLS], I, Love, Paris, [SEP]]

  [I,love,Paris] —Add [CLS],[SEP],[PAD]→ [[CLS], I, Love, Paris, [SEP],[PAD]]

  + When the max length of the token is 6?

01 Embedding Extraction from BERT

- Preprocessing Step 3 : Generation of Attention mask
  - ➢ Set the Attention Mask value to 1 for all (token) locations and 0 for locations with a [PAD] token.

Q) Why do we need an attention mask?

A) To indicate the location of the tokens that hold the actual information

[Attention Mask]

[[CLS], I, Love, Paris, [SEP],[PAD]] → Generate Attention mask → [1,1,1,1,1,0]

01 Embedding Extraction from BERT

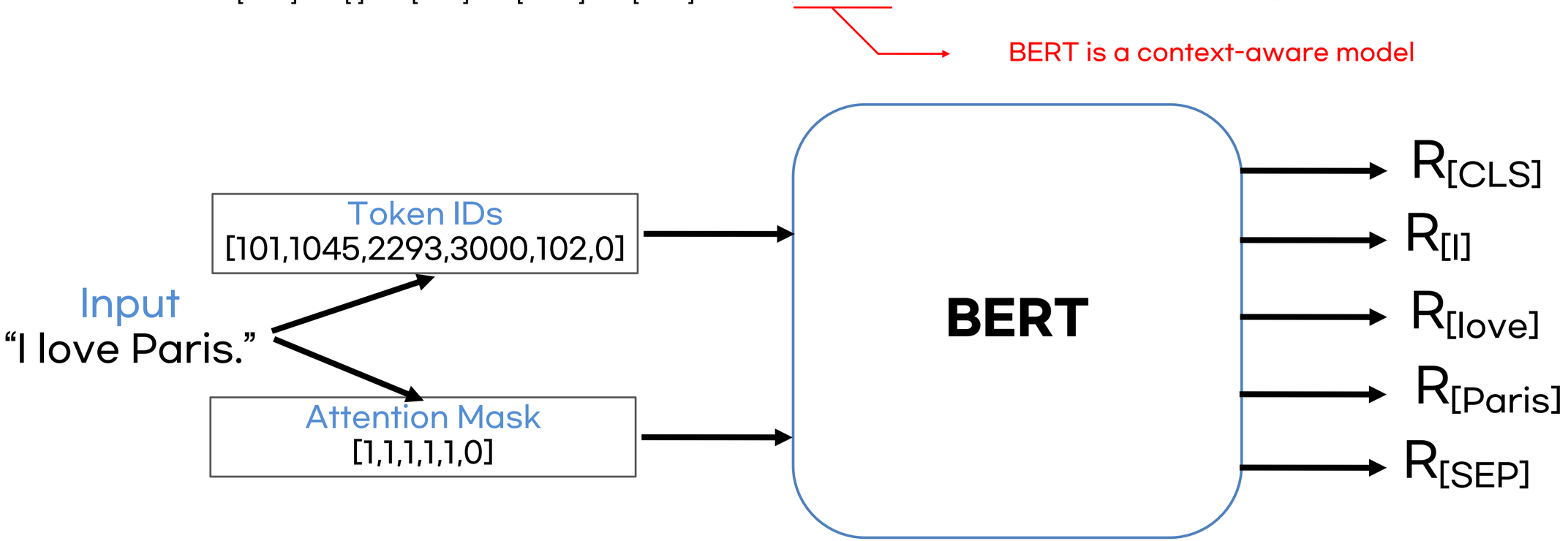
- Preprocessing Step 4 : Token ID mapping
  - ➢ Map each token to a unique token ID. (Token IDs may vary by tokenizer).

[An example of Token ID mapping]

[[CLS], I, Love, Paris, [SEP],[PAD]] —Token ID mapping→ [101,1045,2293,3000,102,0]

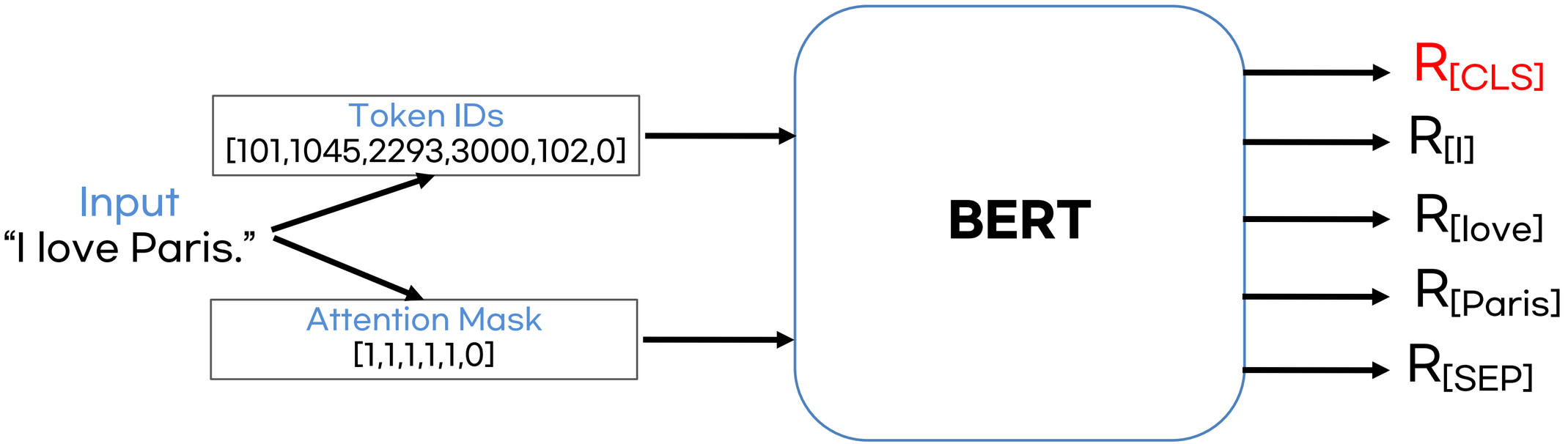
01 Embedding Extraction from BERT

- Overall Process of BERT embeddings
  - By feeding the Token ID and Attention Mask into BERT, get an embedding of each token.
  - $R_{[CLS]}$, $R_{[I]}$, $R_{[love]}$, $R_{[Paris]}$, $R_{[SEP]}$ are contextualized word embeddings



01 Embedding Extraction from BERT

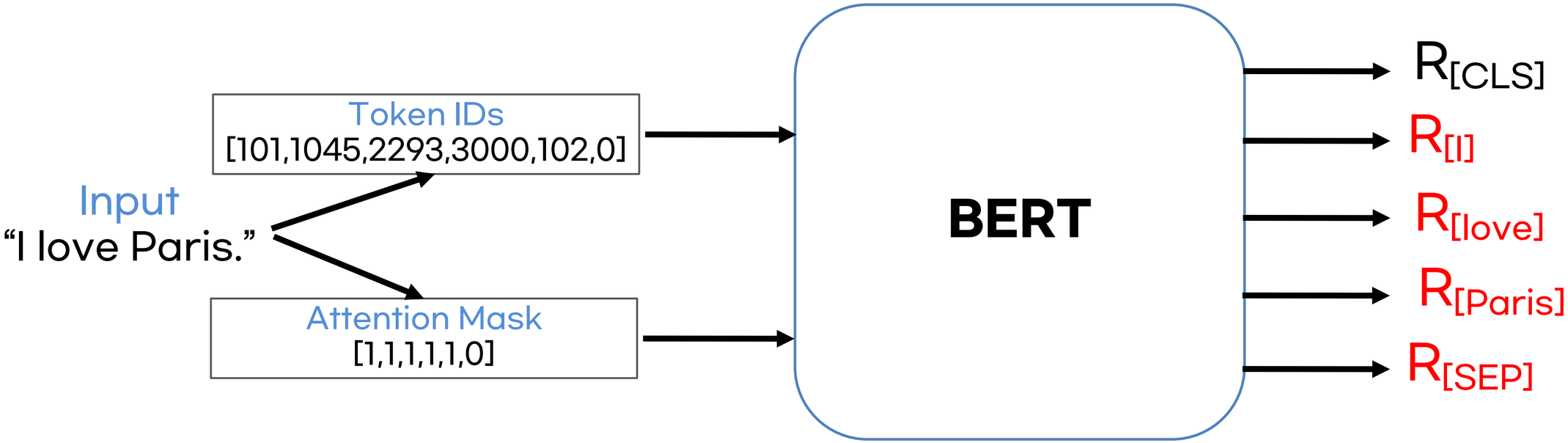
- To get a sentence representation of “I love Paris”
  - $R_{[CLS]}$ has an aggregate representation of the entire sentence.



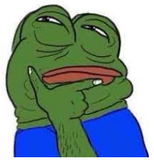
01 Embedding Extraction from BERT

Q) Then is it always a good idea to use $R_{[CLS]}$ for sentence representation?

A) No , Another efficient way is to average/pool the representations of all tokens.



Yeah, yeah, yeah, I get it, but how do we implement something that complicated?



Let’s implement all with Hugging Face

02 BERT using Hugging Face

- HuggingFace
  - A package that distribute pre-trained models, data, and code based on Transformer
  - Very useful and powerful in NLP/NLU tasks
  - It includes thousands of pre-trained models in over 100 languages
  - Works with both Pytorch and TensorFlow

HUGGING FACE

02 BERT using Hugging Face

- 1. Install the transformer package

```
!pip install transformers
```

- 2. Import Classes
  - BertModel : A class for using pre-trained BERT models
  - BertTokenizer : A class for the WordPiece-based BERT tokenizer
  - torch : PyTorch lib

```
from transformers import BertModel,BertTokenizer
import torch
```

02 BERT using Hugging Face

➢ 3. Import a pre-trained BERT

```
1 model = BertModel.from_pretrained('bert-base-uncased')
2 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

➢ bert-base-uncased ?

- HuggingFace's Transformer library has a large number of pre-trained models
- For bert-base-uncased, it can be understood as the uncased model of the bert-base architecture.
- “uncased” : The uncased model makes all tokens lowercase

  ex) I Love Paris > [i, love, paris]
- “cased” : Tokens are not lowercased.

  ex) I Love Paris > [I, Love ,Paris]
- Whether you use "uncased" or "cased" depends on what task you’re handling.

02 BERT using Hugging Face

- Let’s have an example with two sentences below:
  - “I love Paris”
  - “I am Researcher”

- Do the four steps of preprocessing for BERT
  1. Tokenization
  2. Add Special Token
  3. Generate Attention Mask
  4. Token ID mapping

02 BERT using Hugging Face

- Step 1: Tokenization

Input
"I love Paris." → Tokenization → Tokenized one
[I , love , Paris]

```
1 sentence_1= 'I love Paris'
2 sentence_2 = 'I am Resercher'
```

```
1 tokens_1 = tokenizer.tokenize(sentence_1)
2 tokens_2 = tokenizer.tokenize(sentence_2)
3 print(tokens_1,tokens_2,sep='\n')
```

```
['i', 'love', 'paris']
['i', 'am', 'res', '##er', '##cher']
```

02 BERT using Hugging Face

- Step 2: Add Special Token
  - [CLS] : Start
  - [SEP] : End
  - [PAD] : Padding

[I,love,Paris] → Add [CLS],[SEP],[PAD] → [[CLS], I, Love, Paris, [SEP]]

Add [CLS], [SEP] tokens

```
1 # [CLS],[SEP] 토큰 추가하기
2 tokens = ['[CLS]'] + tokens + ['[SEP]']
3 tokens
```

```
['[CLS]', 'i', 'love', 'paris', '[SEP]']
```

Fixed Length 7

```
1 # 고정길이 7 - [PAD] 토큰 추가
2 tokens_1 = tokens_1 + (7-len(tokens_1))*['[PAD]']
3 tokens_2 = tokens_2 + (7-len(tokens_2))*['[PAD]']
4 print(tokens_1,tokens_2,sep='\n')
```

```
['[CLS]', 'i', 'love', 'paris', '[SEP]', '[PAD]', '[PAD]']
['[CLS]', 'i', 'am', 'res', '##er', '##cher', '[SEP]']
```

02 BERT using Hugging Face

➢ Step 3: Generate Attention Mask

[[CLS], I, Love, Paris, [SEP],[PAD]] —Attention mask→ [1,1,1,1,1,0]

```
# 어텐션 마스크 생성
attention_mask_1 = [1 if token !='[PAD]' else 0 for token in tokens_1]
attention_mask_2 = [1 if token !='[PAD]' else 0 for token in tokens_2]
print(attention_mask_1,attention_mask_2,sep='\n')
```

```
[1, 1, 1, 1, 1, 0, 0]
[1, 1, 1, 1, 1, 1, 1]
```

02 BERT using Hugging Face

- Step 4: Token ID mapping
  - It maps each token to a unique Token ID.

[[CLS], I, Love, Paris, [SEP],[PAD]] —Token ID mapping→ [101,1045,2293,3000,102,0]

```
1 # token ID 매핑
2 token_ids_1 = tokenizer.convert_tokens_to_ids(tokens_1)
3 token_ids_2 = tokenizer.convert_tokens_to_ids(tokens_2)
4 print(token_ids_1,token_ids_2,sep='\n')
```

```
[101, 1045, 2293, 3000, 102, 0, 0]
[101, 1045, 2572, 24501, 2121, 7474, 102]
```

02 BERT using Hugging Face

- Step 5: Tensor transformation
  - token_ids1 is a list type, thus transform it as Tensor in Pytorch

```
# 텐서 변환
# unsqueeze 추가하는 이유 -> 배치 차원 추가
token_ids_1 = torch.tensor(token_ids_1).unsqueeze(0)
token_ids_2 = torch.tensor(token_ids_2).unsqueeze(0)

attention_mask_1= torch.tensor(attention_mask_1).unsqueeze(0)
attention_mask_2 =torch.tensor(attention_mask_2).unsqueeze(0)
```

```
|token_ids_1|= torch.Size([1, 7])
|token_ids_2|= torch.Size([1, 7])
|attention_mask_1|= torch.Size([1, 7])
|attention_mask_2|= torch.Size([1, 7])
```

02 BERT using Hugging Face

➢ Step 5: Tensor transformation

- There is also a way to combine them into a batch dimension without using *unsqueeze()*

```
1 token_ids = torch.tensor([token_ids_1,token_ids_2])
2 attention_mask = torch.tensor([attention_mask_1,attention_mask_2])
3
4 print('|token_ids|=',token_ids.shape,'|attention_mask|=',attention_mask.shape)
5 print("token_ids",token_ids,sep='\n',end='\n\n')
6 print("attention_mask",attention_mask,sep='\n')
```

```
|token_ids|= torch.Size([2, 7]) |attention_mask|= torch.Size([2, 7])
token_ids
tensor([[  101,  1045,  2293,  3000,   102,     0,     0],
        [  101,  1045,  2572, 24501,  2121,  7474,   102]])

attention_mask
tensor([[1, 1, 1, 1, 1, 0, 0],
        [1, 1, 1, 1, 1, 1, 1]])
```

02 BERT using Hugging Face

➢ Step 6: Extraction of BERT embeddings

☒ Put "token_ids" and "attention_mask" generated by the preprocessing to extract $R_{[CLS]}$

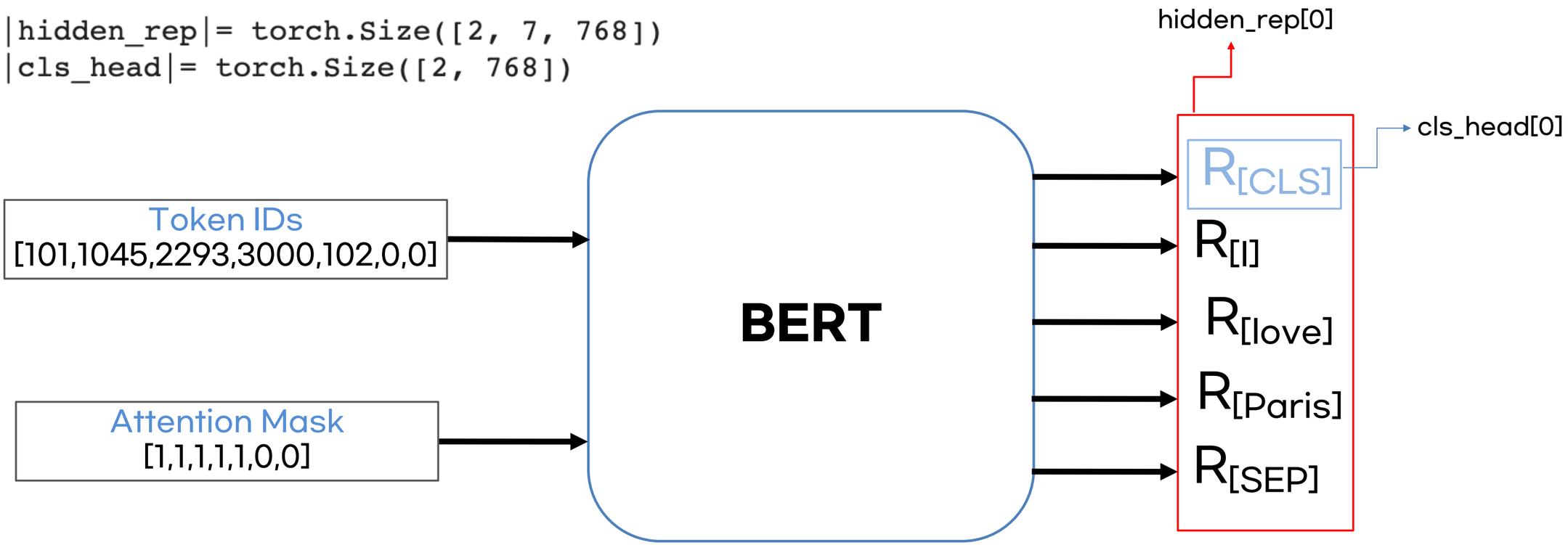
```
# 모델 입력
outputs = model(token_ids,attention_mask=attention_mask)
hidden_rep , cls_head = outputs.last_hidden_state,outputs.pooler_output
```

02 BERT using Hugging Face

- Step 6: Extraction of BERT embeddings

```
1 print("|hidden_rep|=",hidden_rep.shape)
2 print("|cls_head|=",cls_head.shape)
```

```
|hidden_rep|= torch.Size([2, 7, 768])
|cls_head|= torch.Size([2, 768])
```



02 BERT using Hugging Face

- Step 6: Extraction of BERT embeddings
  - |hidden_rep| = torch.size([2, 7, 768])
    - [2, 7, 768] = (batch size (number of sentences), length of token, embedding dimensions)

```
1 print("|hidden_rep|=",hidden_rep.shape)
2 print("|cls_head|=",cls_head.shape)
```

```
|hidden_rep|= torch.Size([2, 7, 768])
|cls_head|= torch.Size([2, 768])
```

02 BERT using Hugging Face

➢ Is the embedding obtained from the final encoder layer the best representation?

- BERT researchers experiment with fetching embeddings from different encoder layers (NER task)

- F1-Score
  - Last layer (h_{12}) < Last 4 layers ($h_{9\ to\ 12}$)

Ref) Getting Started with Google BERT

Layer	Layer in Code	Performance
Word embedding	h_0	91.0
Last previous layer	h_{11}	95.6
Last layer	h_{12}	94.9
Weighted-sum of last four layers	$h_{9\ to}\ h_{12}$	95.9
Concatenation of Last four layers	$h_{9\ to}\ h_{12}$	96.1
Weighted-sum of all layers	$h_{1\ to}\ h_{12}$	95.5

- Of course, using a lot of intermediate encoder layers doesn't guarantee performance, but it does show that the results of the intermediate layers can be used meaningfully.

02 BERT using Hugging Face

➢ 1. Importing a pre-trained model and Tokenizer

```
# BertModel 의 output_hidden_states True로 설정
model = BertModel.from_pretrained('bert-base-uncased',output_hidden_states=True)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

• "output_hidden_state=True" to get the hidden_state value of the intermediate layer

➢ 2. Preprocessing

```
# 입력 전처리
sentence = 'I love Paris'
tokens = tokenizer.tokenize(sentence)
tokens = ['[CLS]'] + tokens + ['[SEP]','[PAD]','[PAD]']
attention_mask = [1 if token != '[PAD]' else 0 for token in tokens]
token_ids = tokenizer.convert_tokens_to_ids(tokens)
token_ids = torch.tensor(token_ids).unsqueeze(0)
attention_mask = torch.tensor(attention_mask).unsqueeze(0)
```

1. tokenization
2. Add Special Tokens
3. Gen Attention mask
4. Token ID mapping
5. Tensor transformation

02 BERT using Hugging Face

➢ Extract embedding from all encoder layers

```
1 outputs = model(token_ids,attention_mask=attention_mask)
2 last_hidden_state= outputs.last_hidden_state
3 pooler_output = outputs.pooler_output
4 hidden_states = outputs.hidden_states
```

- last_hidden_state : Embeddings of the last layer
- pooler_output : An embedding of the first token of the last layer ($R_{[CLS]}$)
- hidden_states : Embeddings of all encoder layers

02 BERT using Hugging Face

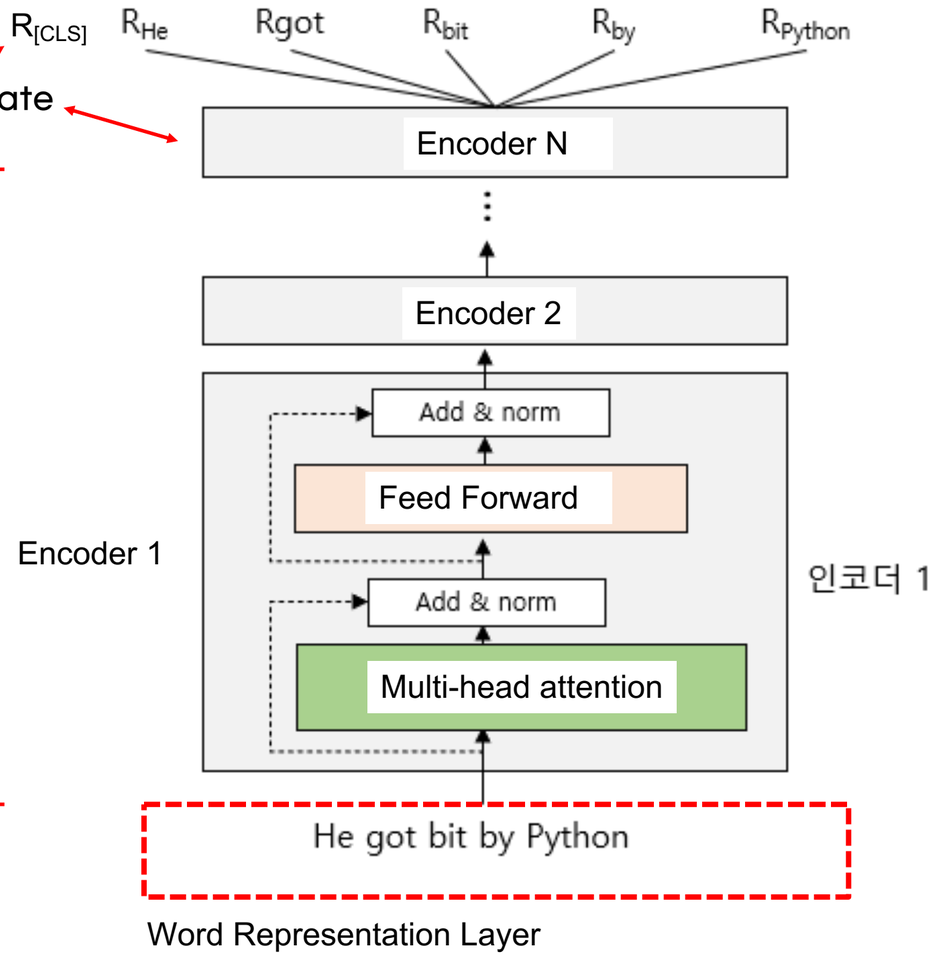
- Extract embedding from all encoder layers

```
|last_hidden_state|= torch.Size([1, 7, 768])
|pooler_output|= torch.Size([1, 768])
|h0| = torch.Size([1, 7, 768])
|h1| = torch.Size([1, 7, 768])
|h2| = torch.Size([1, 7, 768])
|h3| = torch.Size([1, 7, 768])
|h4| = torch.Size([1, 7, 768])
|h5| = torch.Size([1, 7, 768])
|h6| = torch.Size([1, 7, 768])
|h7| = torch.Size([1, 7, 768])
|h8| = torch.Size([1, 7, 768])
|h9| = torch.Size([1, 7, 768])
|h10| = torch.Size([1, 7, 768])
|h11| = torch.Size([1, 7, 768])
|h12| = torch.Size([1, 7, 768])
```

last_hidden_state

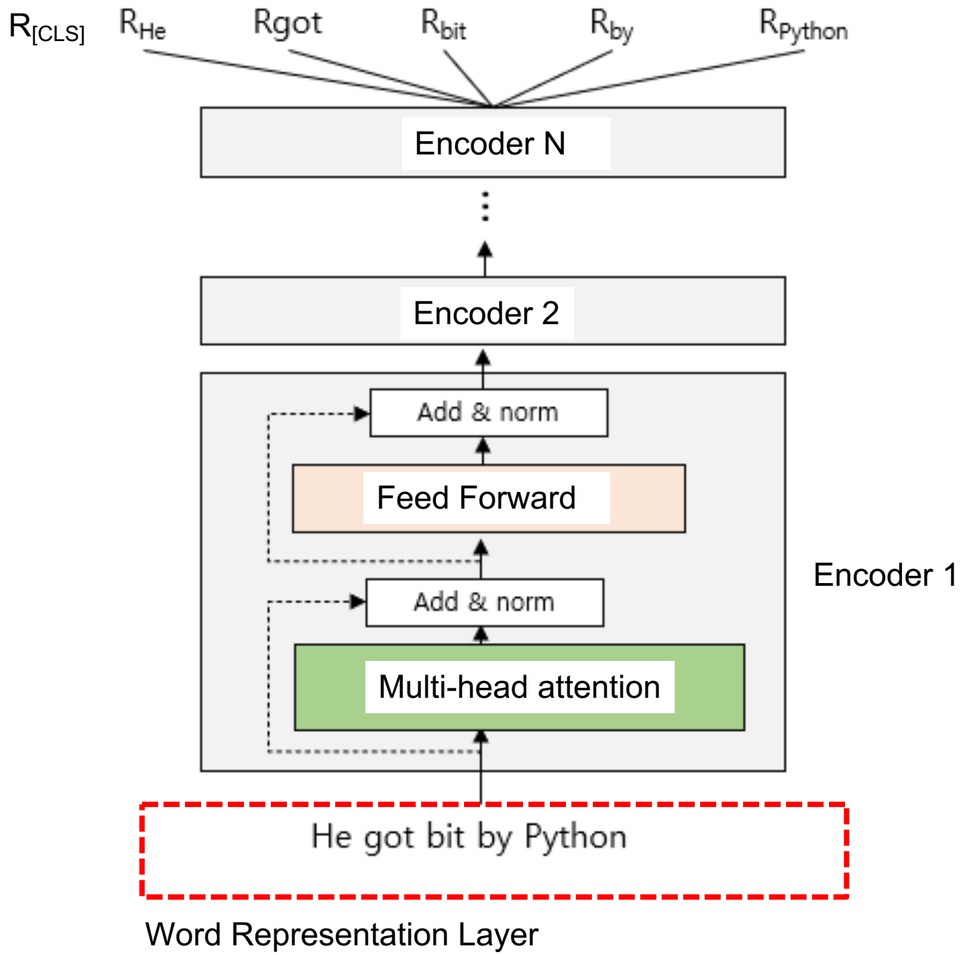
pooler_output

hidden_states



02 BERT using Hugging Face

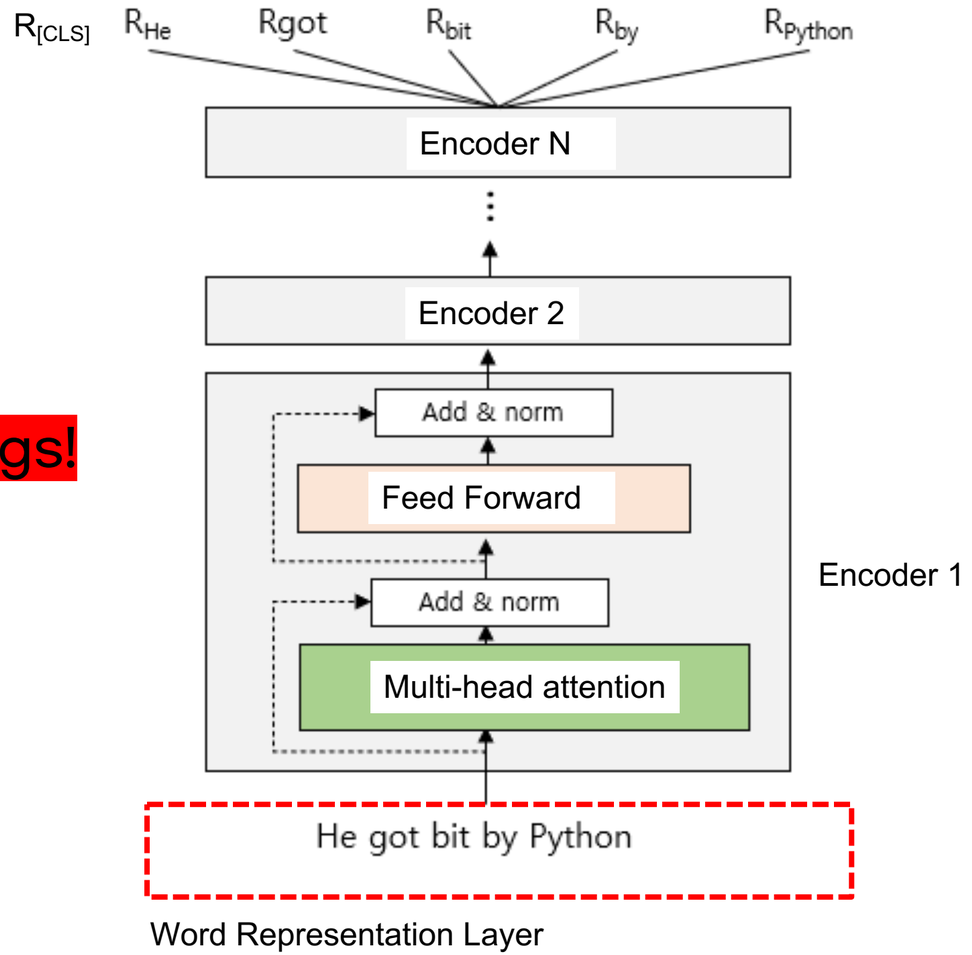
- Extract embedding from all encoder layers
  - Okay, I understand extracting word embeddings from BERT, but how do I do this?



02 BERT using Hugging Face

- Extract embedding from all encoder layers
  - Okay, I understand extracting word embeddings from BERT, but how do I do this?

Just apply it like word embeddings!

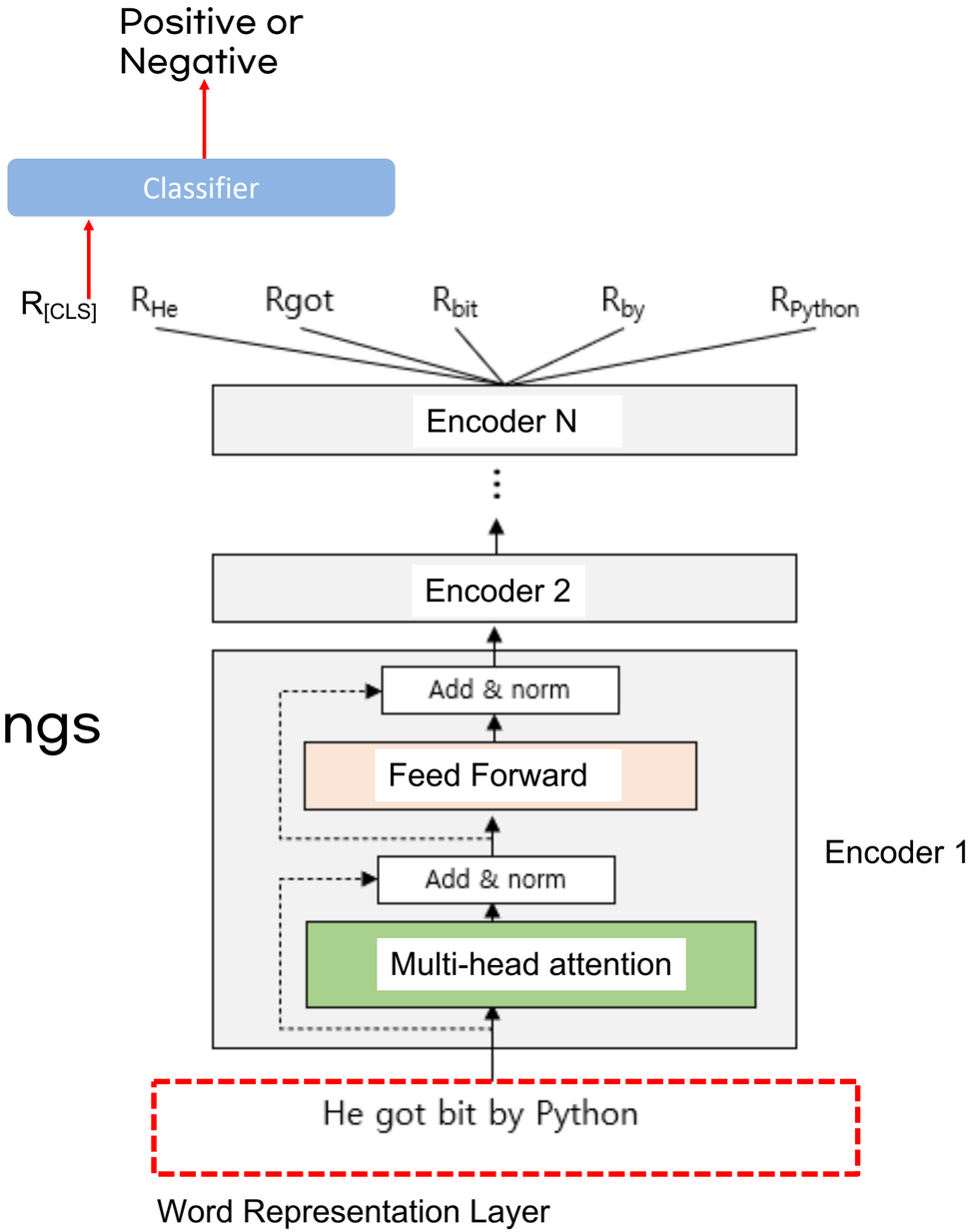


02 BERT using Hugging Face

- Extract embedding from all encoder layers
  - Okay, I understand extracting word embeddings from BERT, but how do I do this?

Just apply it likes word embeddings

with Finetuning



Contents

Huh? The LM understood the grammar and semantics of the language?

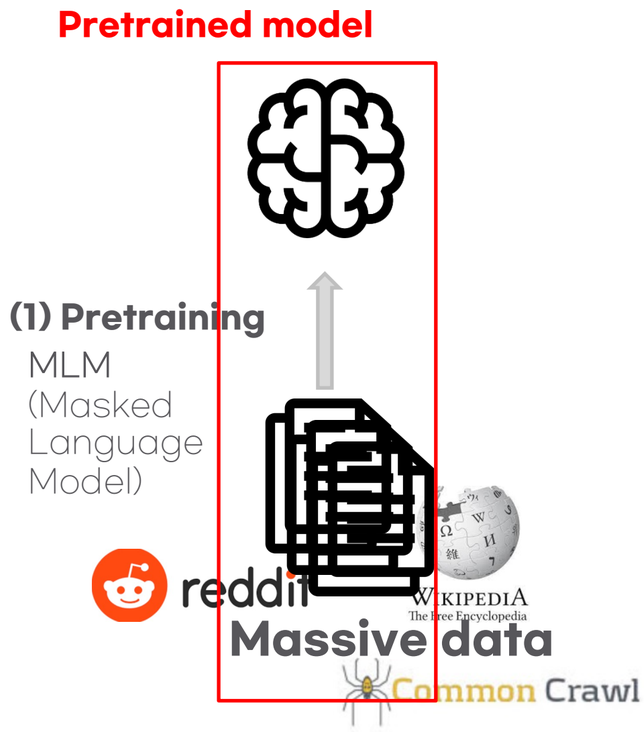
If only we could fine-tune a pretrained LM that has learned grammar and semantics to specific tasks like NLI and machine translation....

But is this possible?

Pretraining and Finetuning

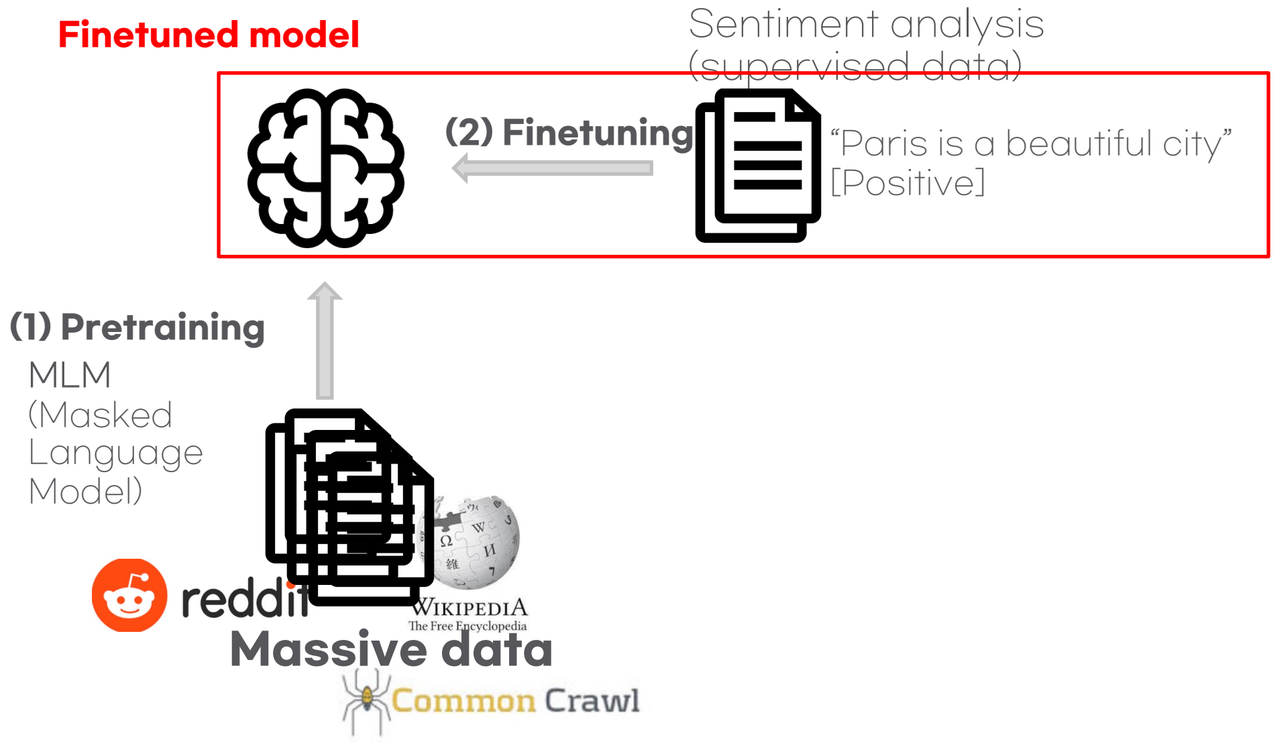
- Pretraining in NLP: Steps to acquire general language knowledge using massive data

  → That's what we've done using BERT



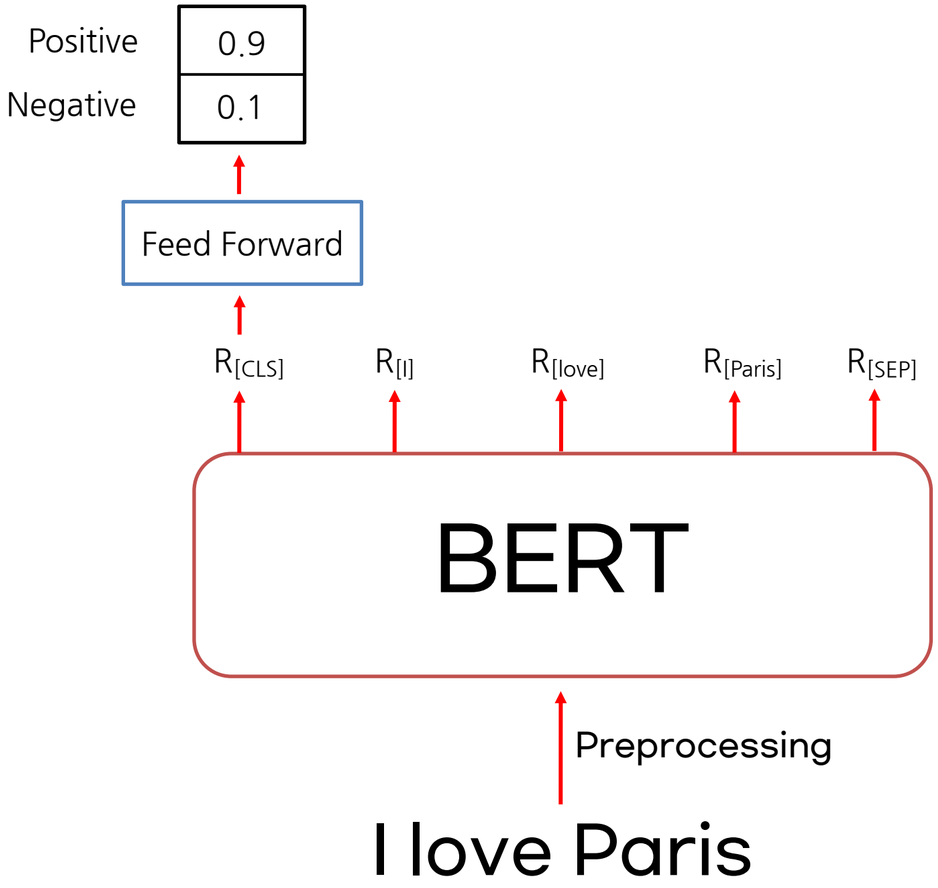
Pretraining and Finetuning

- Pretraining in NLP: Steps to acquire general language knowledge using massive data
- Finetuning in NLP: Steps to tune for a specific task (NER, Translation, QA …)



01 Finetuning for Text Classification

- ❖ Model Architecture
- ❖ Task : Sentiment Analysis (Positive: 1, Negative: 0)



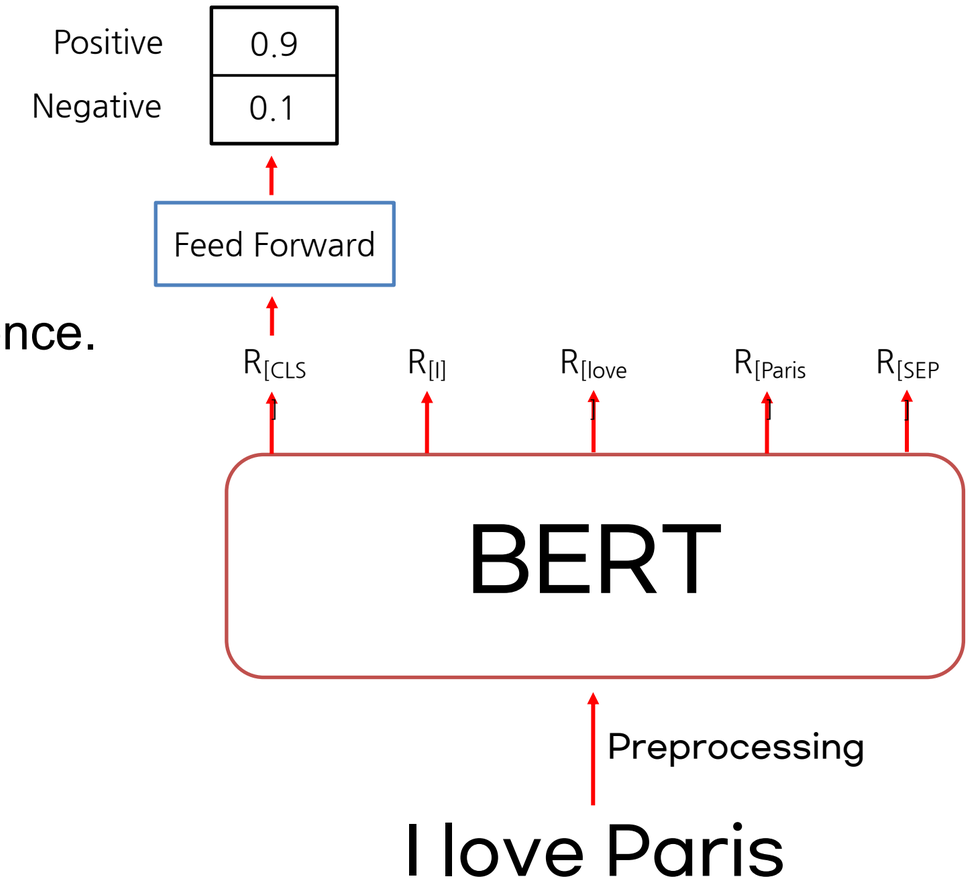
01 Finetuning for Text Classification

❖ Model Architecture

❖ Task : Sentiment Analysis (Positive: 1, Negative: 0)

- Why only put R[CLS] in the feedforward network?

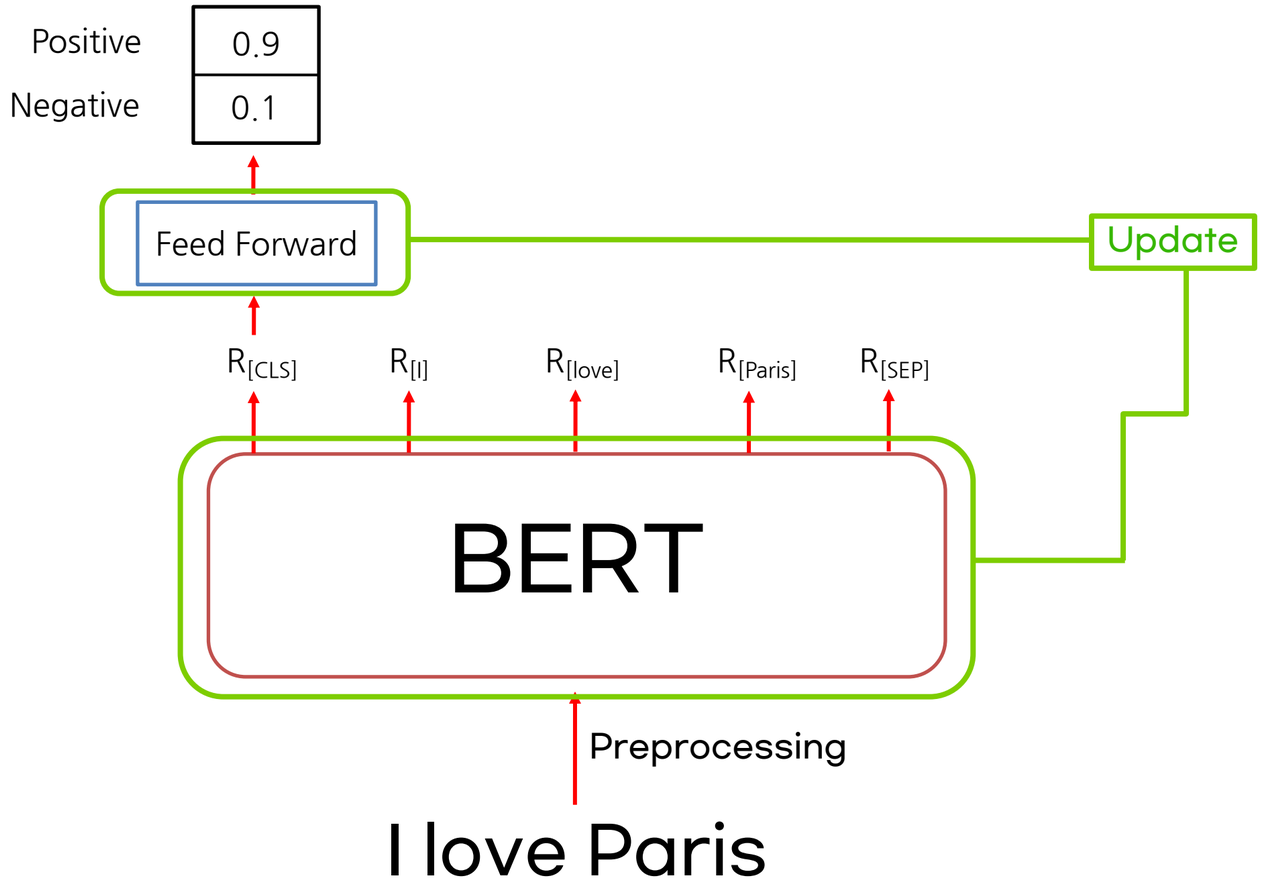
▶[CLS] holds the aggregate representation of the sentence.



01 Finetuning for Text Classification

❖ Fine Tuning Methods (1): Full training

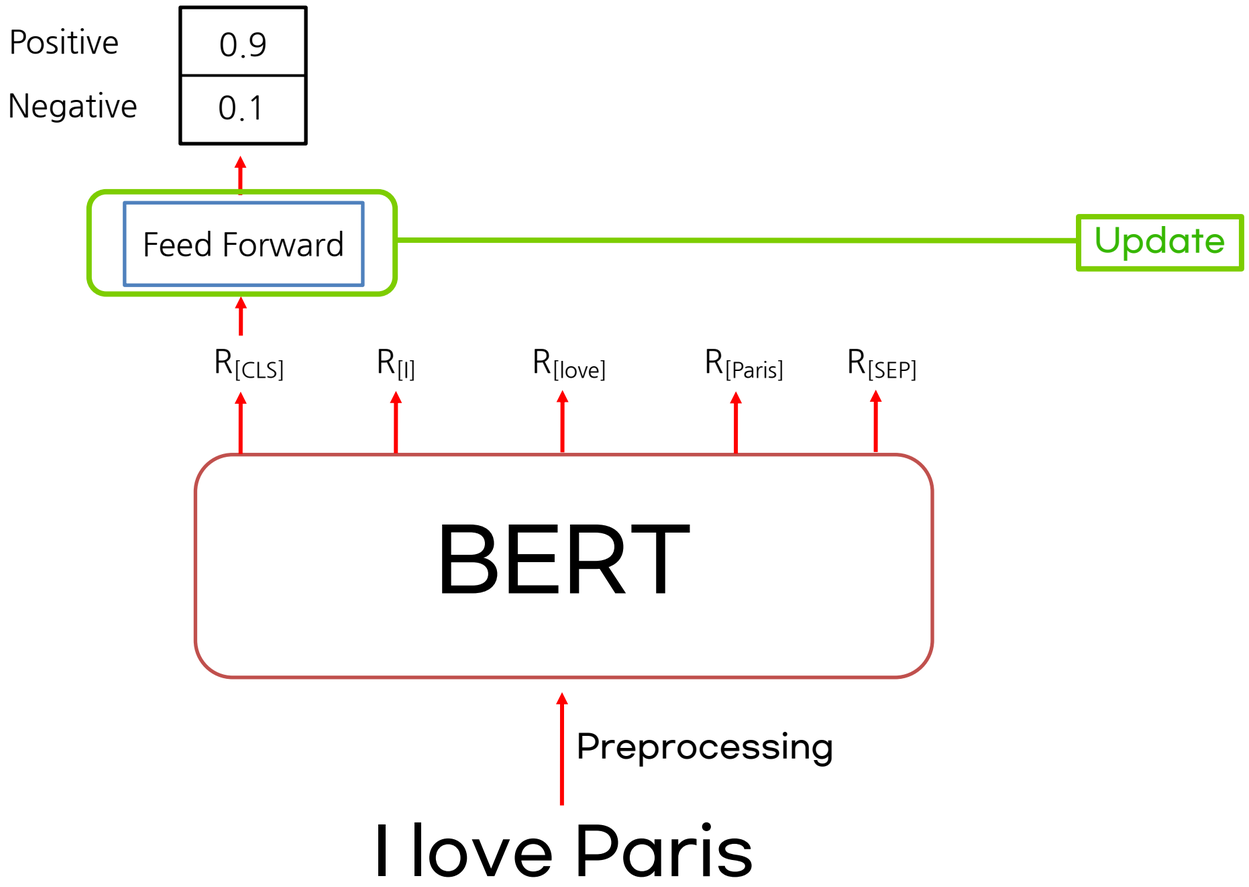
1. Update the weights of the pre-trained BERT model with the classification layer



01 Finetuning for Text Classification

❖ Fine Tuning Methods (1): Local training (Partial training)

1. Update only the weights in the classifier



01 Finetuning for Text Classification

❖ Code

1. Installing the library

```
[13]  1 !pip install nlp
      2 !pip install transformers
```

2. Import modules

```
1 from transformers import BertForSequenceClassification
2 from transformers import BertTokenizerFast
3 from transformers import Trainer
4 from transformers import TrainingArguments
5
6 from nlp import load_dataset
7
8 import torch
9 import numpy as np
```

01 Finetuning for Text Classification

3. Load datasets

   - IMDB datasets

```
!pip install datasets accelerate -qqq

from datasets import load_dataset
dataset = load_dataset('imdb', split="train[:1000]")
```

```
#데이터셋 및 모델 로딩
!gdown https://drive.google.com/uc?id=11_M4ootuT7I1G0RlihcC0cA3Elqotlc-
dataset = load_dataset('csv',data_files='./imdbs.csv',split='train')
```

https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

review	sentiment
49582 unique values	2 unique values
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. The...	positive
A wonderful little production. The filming technique is very unassuming- very old-time-B...	positive
I thought this was a wonderful way to spend time on a too hot summer weekend, sitting in the air con...	positive
Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet	negative

01 Finetuning for Text Classification

4. Train Test Split

```
1 dataset = dataset.train_test_split(test_size=0.3)
```

```
1 train_set = dataset['train']
2 test_set = dataset['test']
```

01 Finetuning for Text Classification

5. Load a pre-trained BERT model and tokenizer

```
1 model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
2 tokenizer = BertTokenizerFast.from_pretrained('bert-base-uncased')
```

6. Preprocessing
    1. Tokenization
    2. Add Special Token
    3. Generate segment tokens
    4. Generate Attention Mask
    5. Token ID mapping

Should I do this again?? No!!!! I'm going to drop the class.

01 Finetuning for Text Classification

- BertTokenizerFast performs this input preprocessing in a single pass.

```
1 tokenizer('I love Paris')
```

```
{'input_ids': [101, 1045, 2293, 3000, 102], 'token_type_ids': [0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1]}
```

- Preprocessing
  1. Tokenization
  2. Add Special Token
  3. Generate segment tokens
  4. Generate Attention Mask
  5. Token ID mapping

BertTokenizerFast

01 Finetuning for Text Classification

- BertTokenizerFast also adds a 'padding' parameter to append a [PAD] token to the max_length of the to the max_length.

```
1 tokenizer('I love Paris',padding='max_length',max_length=7)
```

```
{'input_ids': [101, 1045, 2293, 3000, 102, 0, 0], 'token_type_ids': [0, 0, 0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1, 0, 0]}
```

01 Finetuning for Text Classification

- Preprocessing over the dataset
  - preprocess method
  - Perform input preprocessing on the train/test data using the map method.

```
def preprocess(data):
  return tokenizer(data['text'],padding='max_length',max_length=10,truncation=True)
```

```
train_set = train_set.map(preprocess,batched=True,batch_size=len(train_set))
test_set = test_set.map(preprocess,batched=True,batch_size=len(test_set))
```

  - Use the “set_format” method to enter the required columns and the required format for your dataset.

```
train_set.set_format('torch',columns=['input_ids','attention_mask','label'])
test_set.set_format('torch',columns=['input_ids','attention_mask','label'])
```

01 Finetuning for Text Classification

- 7. Train Model using Trainer in HuggingFace
  - With hyperparameters
    - batch_size=8 , epochs=2, warmup_steps=500, weight_decay=0.01
  - With TrainingArguments
    - A helper class to set your hyperparameters and options

```
training_args = TrainingArguments(
    output_dir = './results',          → model save path
    num_train_epochs=epochs,           → # of epoch
    per_device_train_batch_size=batch_size,   → Size of batch
    per_device_eval_batch_size=batch_size,    → Size of batch for evaluation
    warmup_steps=warmup_steps,         → #warmup step
    weight_decay=weight_decay,         → weight decay
    evaluation_strategy='steps',       → Evaluation strategy during training ('no' : without evaluation , 'steps': every step, 'epochs': every epoch)
    logging_dir='./logs'               → log save path
)
```

01 Finetuning for Text Classification

- 7. Train Model using Trainer in HuggingFace
  - Set Trainer instance from HuggingFace

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_set,
    eval_dataset=test_set
)
```

  - Model train and eval using Trainer instance

```
trainer.train()
```

```
trainer.evaluate()
```

02 Finetuning for Natural Language Inference

❖ Task description

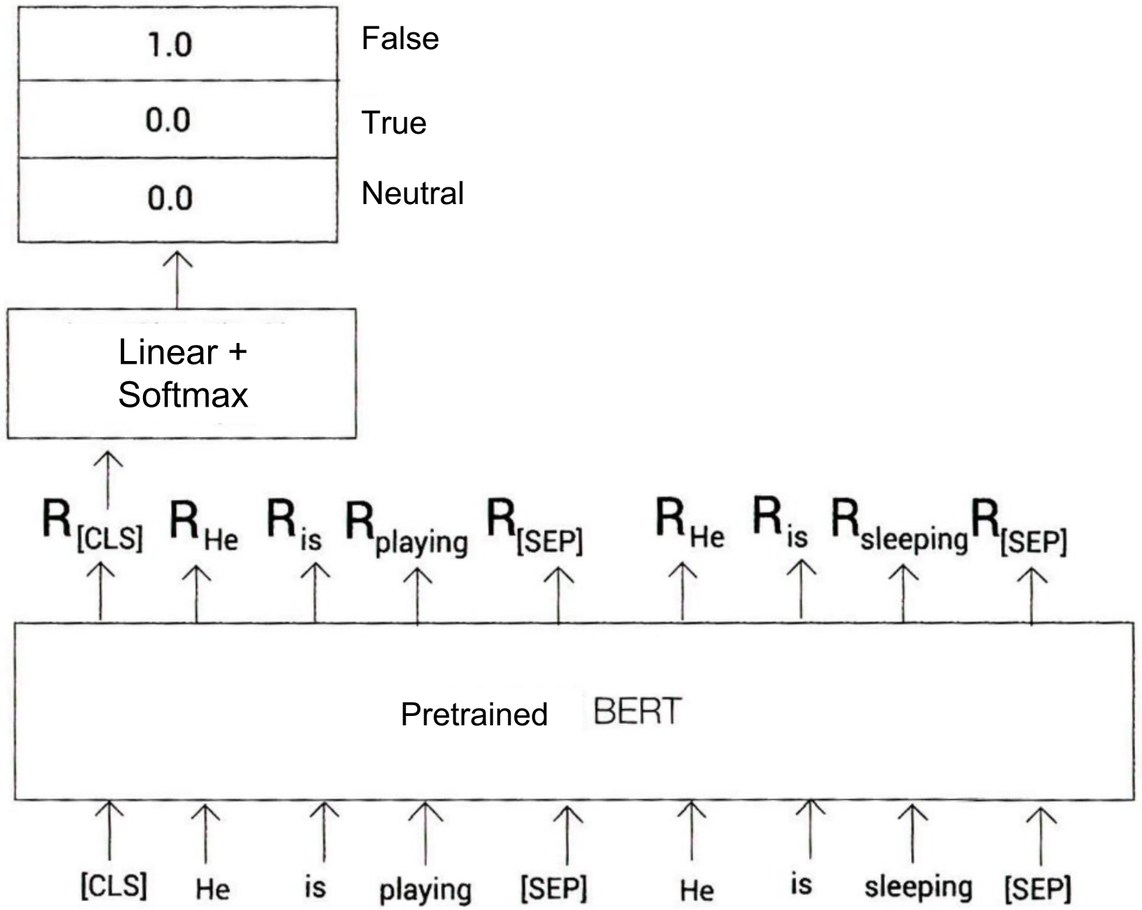
▶ Natural language inference (NLI) is the task of determining whether a premise is true, false, or neutral given a hypothesis.

Premise	Hypothesis	Label
He’s playing	He's sleeping.	False
여러 남성이 플레이하는 축구 게임	몇몇 남자들이 스포츠를 하고 있다	True
웃고 있는 노인과 청년	두 남자가 바닥에서 놀고 있는 강아지들을 보고 웃고 있다	Neutral

Ref) Getting Started with Google BERT

02 Finetuning for Natural Language Inference

❖ Model Architecture



Ref) Getting Started with Google BERT

03 Finetuning for Question-Answering

❖ Task description

- Task to extract answers from paragraphs for a given question
- Input : Question-Paragraph Pair
- The BERT should return a range of text corresponding to the response in the Paragraph

◆ Question: “What is the immune system?”

Paragraph: “The immune system is a system of various biological structures and processes within an organism that protects against disease. To function properly, the immune system must detect various substances known as pathogens, from viruses to parasites, and distinguish them from the healthy tissues of an organism.”

Answer : “a system of various biological structures and processes within an organism that protects against disease”

03 Finetuning for Question-Answering

❖ Key Point for QA!

- To solve a QA, we need the start and end indices of the text range that contains the answer to a given paragraph.
- Use the probabilities of the start and end tokens of the answer to find it.
- We will check whether which token is the best possible START and END tokens by classifier

◆Question: “What is the immune system?”

Paragraph: “The immune system is a system of various biological structures and processes within an organism that protects against disease. To function properly, the immune system must detect various substances known as pathogens, from viruses to parasites, and distinguish them from the healthy tissues of an organism.”

03 Finetuning for Question-Answering

❖ For example

◆ Question: “What is the immune system?”

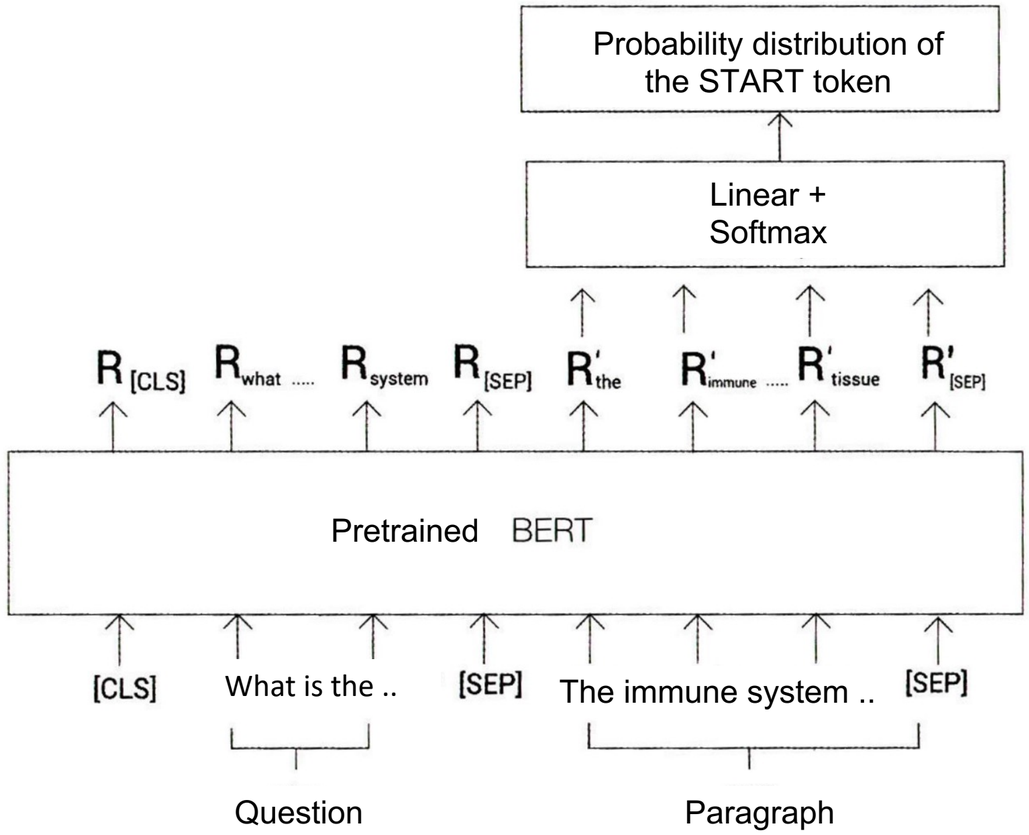
Paragraph: “The immune system is a system of various biological structures and processes within an organism that protects against disease. To function properly, the immune system must detect various substances known as pathogens, from viruses to parasites, and distinguish them from the healthy tissues of an organism.”

start logits-Para: 0.143, 1.234 3.224 -6.224 **12.34** ….”

softm-Para: 0.025, 0.112 0.242 0.002, **0.834** ….”

03 Finetuning for Question-Answering

❖ Model Architecture



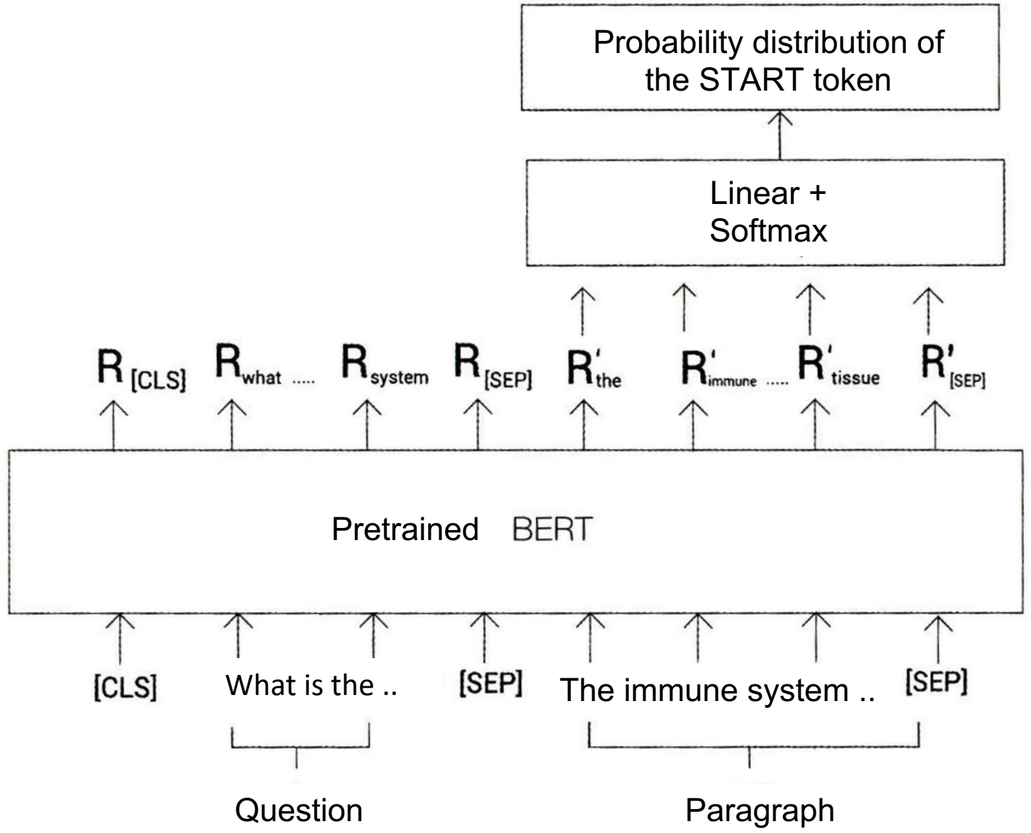
03 Finetuning for Question-Answering

❖ Training Procedure

1. Tokenize question-paragraph pairs

2. Input tokens into a pre-trained BERT , returning an embedding of all tokens

3. Classification for the START and END tokens using the BERT embeddings



03 Finetuning for Question-Answering

- Import Modules

```
1 from transformers import BertForQuestionAnswering,BertTokenizerFast
```

- Load the pretrained model and tokenizer

```
1 model = BertForQuestionAnswering.from_pretrained('bert-large-uncased-whole-word-masking-finetuned-squad')
2 tokenizer = BertTokenizerFast.from_pretrained('bert-large-uncased-whole-word-masking-finetuned-squad')
```

03 Finetuning for Question-Answering

- Preprocessing

Add Special Token

```
question = '[CLS]' + question + '[SEP]'
paragraph = paragraph + '[SEP]'
```

Tokenization

```
question_tokens = tokenizer.tokenize(question)
paragraph_tokens = tokenizer.tokenize(paragraph)
```

Concatenation Q and P
Token ID mapping
Generate segment ID
Generate Attention_mask

```
tokens = question_tokens + paragraph_tokens
input_ids = tokenizer(tokens)['input_ids']
segment_ids = tokenizer(tokens)['token_type_ids']
attention_mask = tokenizer(tokens)['attention_mask']
```

Tensor transformation

```
segment_ids = [0] * len(question_tokens)
segment_ids += [1] * len(paragraph_tokens)
```

03 Finetuning for Question-Answering

- Prediction
  - Input input_ids,segment_ids into a model that returns the starting and ending scores for all
  - Return the start/end score for all tokens in the paragraph in the output

```
outputs= model(input_ids,token_type_ids=segment_ids)
start_score = outputs.start_logits
end_score = outputs.end_logits
```

```
start_index = torch.argmax(start_score)
end_index = torch.argmax(end_score)
```

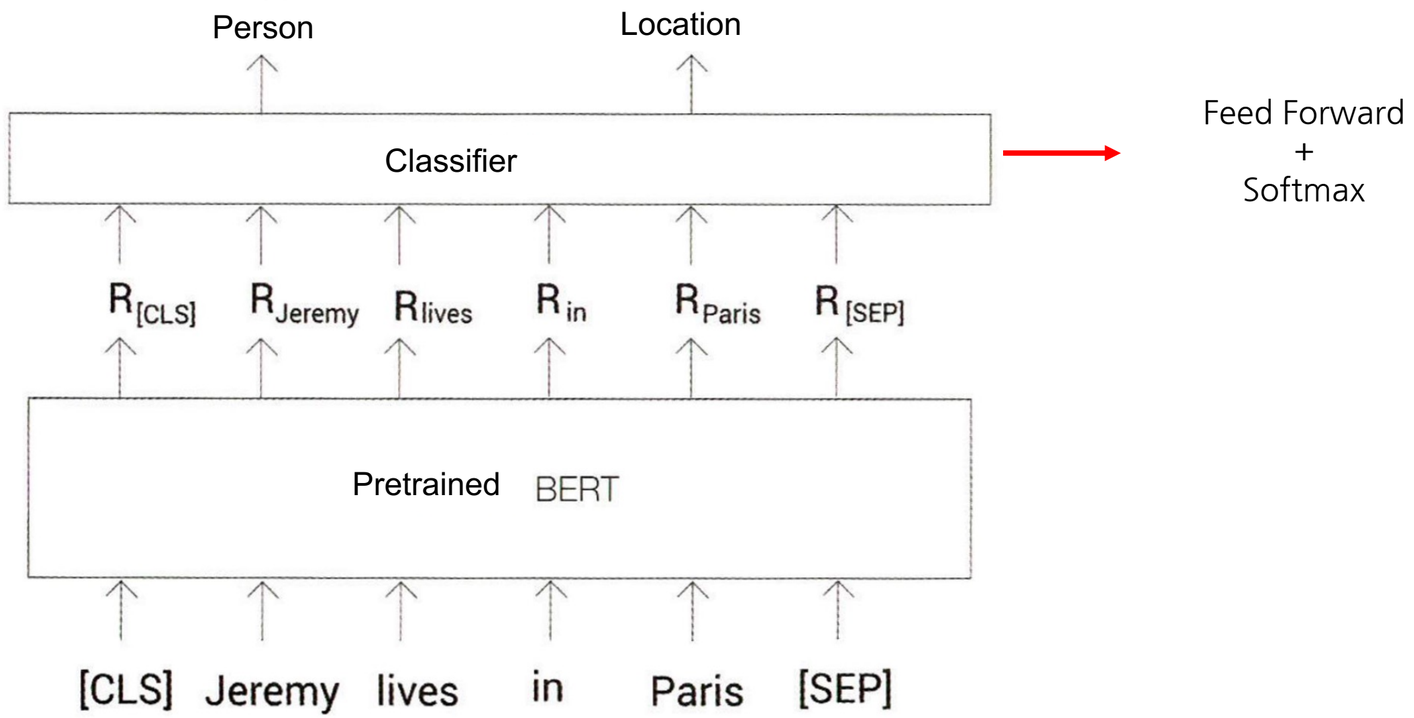
```
text = ' '.join(tokens[start_index:end_index+1])
text = text.replace(' ##','')
print(text)
```

```
질병으로부터 보호하는 유기체 내의 [UNK] 생물학적 구조와 과정의 시스템입니다
```

04 Finetuning for Named Entity Recognition

❖ Task Description

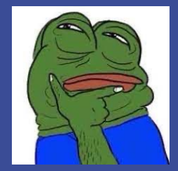
- Named Entity Recognition?
  - ▶ Tasks that categorize object names into predefined categories



Ref) Getting Started with Google BERT

BERT Batch Processing & GPU Parallelism

(Why we should use fixed number of tokens?)



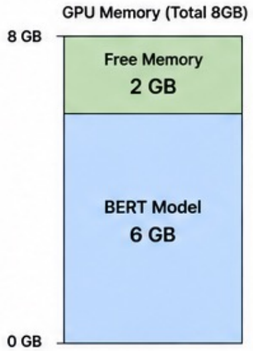
Appendix: BERT Batch Processing & GPU Parallelism

1 BERT Batch Processing & GPU Parallelism

Let's understand with Batch Size = 2

When the GPU processes multiple samples at once, it becomes much faster and more efficient!

2 GPU Memory Situation (Assumption)



Extra Memory required per input sample ≈ 1 GB (forward/backward)

Maximum number of samples processed simultaneously = 2 (Batch Size = 2)

In practice, memory is also used by parameters, activations, attention, gradients, optimizer states, etc.

3 Batch Size ≠ Concatenating Tokens

Wrong: Concatenating Tokens

Sample A (512 tokens)

A_1	A_2	...	A_{511}	A_{512}

Sample B (512 tokens)

B_1	B_2	...	B_{511}	B_{512}

1024 tokens (single sequence)

A_1	A_2	...	A_{512}	B_1	B_2	...	B_{512}

The model will treat it as one long sequence, mixing two sentences! (Attention crosses the boundary)

Right: Batch Processing (Parallel)

Sample A (512 tokens)

A_1	A_2	...	A_{511}	A_{512}

Sample B (512 tokens)

B_1	B_2	...	B_{511}	B_{512}

Two sentences are kept independent and processed simultaneously!

4 Actual Input Tensor Structure

Example: Batch Size = 2, Sequence Length = 512

Input Tensor Shape

[2, 512]

Batch Dimension (number of samples)

Sequence Length (number of tokens)

Token Position

	1	2	3	...	510	511	512
Sample A	A_1	A_2	A_3	...	A_{510}	A_{511}	A_{512}
Sample B	B_1	B_2	B_3	...	B_{510}	B_{511}	B_{512}

Each sample occupies one row in the batch dimension.

5 How BERT Works Internally (Parallel Computation)

BERT is based on large-scale matrix operations!

Input [2, 512] → Token Embedding + Position Embedding → Transformer Encoder (12 layers) → Output

The hidden states have shape [2, 512, 768] (BERT-base, hidden dim = 768)

[2, 512, 768]

Batch Size (2) | Sequence Length (512) | Hidden Dimension (768)

- The 2 samples are processed in parallel using the GPU at every layer!
- Each sample is computed independently, but the operations run simultaneously.

6 CPU Processing vs GPU Batch Parallelism

CPU Style (Sequential)

Sample A → BERT

Sample B → BERT

Sample C → BERT

⋮

Processed one by one

GPU Batch Parallelism (Simultaneous)

Sample A, Sample B → BERT (Batch = 2) → Output A, Output B

Multiple samples are processed together at once!

7 Why Use a Larger Batch Size?

Advantages

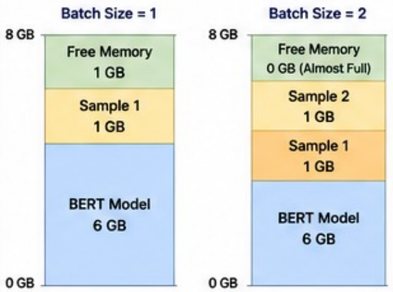
- Higher GPU Utilization
- Higher Throughput
- Faster Training
- Better Computational Efficiency

Disadvantages

- More GPU Memory Usage
- Risk of OOM (Out of Memory)
- More Activation Storage

Finding the right batch size is very important!
(Too small → less efficient, Too large → out of memory)

8 Memory Usage Example (Illustration)



Key Point

The model (6GB) takes a large portion of the memory. The remaining memory determines how many samples we can load simultaneously.

If we increase the batch size, more memory is required.

9 Summary

1. BERT processes multiple sentences in a batch (not by concatenating tokens).
2. Batch Size = 2 means the input tensor shape is [2, 512] or [2, 512, hidden_dim].
3. The GPU processes multiple samples in parallel, not sequentially.
4. GPUs are designed for large matrix operations to maximize efficiency.
5. Larger Batch Size improves speed, but requires more GPU memory.

Batch Size ↔ GPU Memory ↔ Parallel Processing
Understanding this relationship is the key!

Thank you!